**UNIVERSIDAD DE ANTIOQUIA**
**FACULTAD DE INGENIERÍA**
**PROFESOR: ANÍBAL JOSÉ GUERRA**
**INTEGRANTES: MIGUEL ÁNGEL UPEGUI LONDOÑO. DAVID ALEJANDRO MARÍN PINZÓN.**

# INFORME PRELIMINAR. ESTRUCTURA DE DATOS, ANÁLISIS DEL PROBLEMA Y DISEÑO.

## CONTEXTUALIZACIÓN

Este desafío consiste en diseñar e implementar la plataforma ude@Book en C++, mediante el paradigma de programación orientada a objetos (POO). El sistema ofrecerá funcionalidades básicas de una red social, como el inicio de sesión, la administración de amistades, la consulta de sugerencias, la visita de perfiles y álbumes, y el envío y lectura de mensajes. Para ello se emplearán clases, constructores, destructores y memoria dinámica, procurando una solución modular que cumpla las restricciones establecidas en el enunciado.

## ESTRUCTURA DE DATOS

La estructura de datos se organizará mediante clases y estructuras dinámicas de creación propia, destinadas a representar los usuarios y sus relaciones dentro de la plataforma. Entre la información que se deberá administrar se encuentran los datos de cada usuario, sus amigos, las solicitudes de amistad, los mensajes enviados y recibidos, y los álbumes y fotografías asociados a los perfiles. Las relaciones entre estos elementos se implementarán con memoria dinámica y apuntadores, evitando el uso de contenedores de la STL y de estructuras prohibidas por el enunciado.

El programa contará además con un bloque encargado de leer, analizar y distribuir la información de los archivos de entrada en las variables y objetos correspondientes. Se diseñará un formato de almacenamiento permanente para conservar los datos de la plataforma. Cuando el usuario regrese al menú principal, se ejecutará el método de actualización de archivos, encargado de guardar los cambios realizados durante las operaciones. La lectura y escritura de archivos se mantendrán separadas del procesamiento interno de los datos.

## ANÁLISIS

El problema requiere administrar una cantidad potencialmente grande de usuarios y las relaciones entre ellos, así como permitir consultas y modificaciones sin perder la información almacenada. Por esta razón, el diseño debe separar la representación de los datos, las operaciones de la plataforma y la interacción por consola. Esta separación facilitará la implementación, las pruebas y el mantenimiento de cada funcionalidad.

La solución va a organizar el programa en varios archivos; emplear memoria dinámica en el núcleo del sistema; no utilizar herencia, contenedores de la STL, estructuras struct ni bibliotecas externas que resuelvan las estructuras esenciales. También será necesario evitar copias innecesarias de objetos y datos estructurados, favorecer el uso de referencias cuando corresponda y liberar correctamente la memoria reservada.

Además de implementar las funcionalidades solicitadas, el programa deberá estimar y mostrar, al finalizar cada una, la cantidad de iteraciones realizadas y el consumo de memoria de las estructuras, objetos, variables locales y parámetros involucrados. Estas mediciones deberán integrarse al diseño desde el comienzo, para que las operaciones puedan reportar información coherente sobre sus recursos.

## DISEÑO

La propuesta se desarrollará de forma modular. Como punto de partida, se contemplan clases para representar usuarios y mensajes, además de componentes para administrar las amistades, los álbumes y las fotografías, y una clase o módulo central encargado de coordinar las funcionalidades de ude@Book. La distribución definitiva de responsabilidades y las relaciones entre clases se establecerán en el diagrama De clases que deberá reflejar los atribtos, métodos, constructores, destructores y relaciones necesarios.

Las operaciones principales se organizarán en módulos independientes: carga y actualización de archivos; autenticación y selección de fotos de perfil de amigos; envío y aceptación de solicitudes; consulta de perfiles y álbumes; generación de sugerencias de amistad; administración de mensajes; y medición de recursos. Para las sugerencias se analizará una representación de las relaciones de amistad que permita calcular los amigos en común y ordenar los resultados. Para los mensajes se contemplarán el envío, la lectura de mensajes no leídos y la búsqueda por un rango de fechas.

La gestión de memoria será responsabilidad explícita de las clases que reserven recursos dinámicos. Sus constructores inicializarán los atributos y recursos necesarios, mientras que los destructores liberarán la memoria que corresponda. Se evaluará el uso de constructores de copia, getters, setters y sobrecarga de al menos dos operadores, de acuerdo con los requisitos del desafío. Las decisiones definitivas se documentarán y ajustarán durante la implementación.

## FLUJO GENERAL DEL PROGRAMA

Al iniciar la ejecución, el sistema leerá los archivos de almacenamiento permanente y construirá las estructuras dinámicas necesarias para representar los datos. Luego solicitará las credenciales del usuario y verificará su identidad. Tras el ingreso, presentará el menú principal y las fotos de perfil de amigos seleccionadas según las reglas de prioridad indicadas en el enunciado.

Desde el menu el usuario podrá administrar solicitudes de amistad, visitar perfiles y álbumes, consultar sugerencias y gestionar mensajes. Al terminar cada funcionalidad, se mostrarán las métricas de iteraciones y memoria consumida. Cuando el usuario regrese al menú principal, se actualizarán los archivos permanentes con los cambios correspondientes. Finalmente, al salir del programa, se liberarán los recursos dinámicos utilizados.